1.1 Actividad 1: relaciones a partir del enunciado

1. ¿Qué relaciones deberían incorporarse al diagrama? Indique las clases involucradas, el tipo de relación y justifique su elección. Se deben incorporar tres relaciones principales. Primero, una asociación entre las clases *Prestamo* y *Socio* porque el caso indica explícitamente que el préstamo guarda al socio y este se mantiene en el registro incluso después de devolver el libro. Segundo, es necesario agregar otra asociación entre *Prestamo* y *Libro* por la misma razón, ya que el libro queda guardado en el historial del préstamo. Por último, corresponde incluir una dependencia entre *Prestamo* e *Impresora*. Esto ocurre porque la impresora solo se utiliza en el momento para sacar el comprobante y el texto aclara que el préstamo no la guarda en su estructura.

2. Para cada asociación propuesta, indique sus multiplicidades y explique qué significan en el caso.

Para ambas asociaciones la multiplicidad es de 1 a 1 . En el caso de *Prestamo* con *Socio*, esto significa que cada registro de préstamo le pertenece a un único socio involucrado en esa transacción específica. Para *Prestamo* con *Libro*, quiere decir que cada préstamo particular está vinculado a un solo libro prestado.

3. ¿Bastaría representar todas las relaciones como dependencias para describir el caso? Justifique con el enunciado. No bastaría con usar únicamente dependencias. Las dependencias sirven para modelar elementos que se utilizan de forma temporal, como ocurre con el uso de la impresora en este sistema. Sin embargo, el enunciado detalla que el préstamo guarda al socio y al libro, y que además ambos siguen registrados en el historial al realizar la devolución. Para representar que esa información se conserva de forma permanente se requiere el uso de asociaciones.

4. ¿Qué objetos debería guardar Prestamo como atributos? Justifique. La clase *Prestamo* debe guardar un objeto de tipo *Socio* y otro de tipo *Libro*. Esto se justifica directamente en los requerimientos, dado que es obligatorio mantener la información de la persona que solicitó el libro y el ejemplar específico para conservarlos en el historial del sistema.

5. ¿Qué parámetros y tipos propondría para devolver? Explique para qué sirve cada uno. Se proponen dos parámetros para el método encargado de la devolución. El primero es de tipo entero, como por ejemplo un *int diasAtraso*, el cual sirve para calcular la multa aplicando el cobro de $500 por cada día extra mencionado en el texto. El segundo parámetro es un objeto de la clase *Impresora*, como *Impresora impresora*, que permite generar el comprobante utilizando el equipo disponible en ese momento sin necesidad de guardarlo como atributo dentro de la clase.

1.2 Actividad 2: relaciones en el código

1. ¿Dónde se reflejan las asociaciones y dependencias en el código? Señale los atributos o métodos, las clases involucradas y justifique cada relación. Las asociaciones se reflejan claramente en los atributos de la clase *Prestamo*, específicamente en las definiciones de *private Socio socio* y *private Libro libro*. Al tratarse de variables de instancia, estas líneas demuestran que el préstamo conserva dicha información de manera estructural en el tiempo. Por su parte, la dependencia se evidencia en la firma del método *devolver(int diasAtraso, Impresora impresora)*. En este punto, la clase *Impresora* ingresa como parámetro exclusivamente para ejecutar la acción de imprimir y luego se descarta, sin quedar almacenada como un atributo del préstamo.

2. ¿Cuántas asociaciones y dependencias hay entre las cuatro clases? Se identifican en total 3 relaciones dentro del código. Estas corresponden a 2 asociaciones (la de *Prestamo* con *Socio* y la de *Prestamo* con *Libro*) y 1 dependencia (entre *Prestamo* e *Impresora*).

3. ¿Existe una relación directa entre Socio y Libro? Justifique con el código. No existe una relación directa entre ambas clases. Al revisar el código de *Socio* y de *Libro*, se observa que ninguna posee atributos del tipo de la otra clase ni reciben objetos de la otra clase como parámetros en sus métodos. Ambos elementos solo se relacionan de forma indirecta al momento de instanciar un nuevo objeto *Prestamo*.

4. ¿La clase Prestamo aplica SRP? Justifique según sus razones para cambiar y proponga una separación si la considera necesaria. La clase *Prestamo* no respeta el principio SRP al acumular múltiples responsabilidades. Esto se evidencia porque la clase tendría motivos para cambiar si se modifican las reglas internas de devolución del libro, si varía la lógica financiera de la multa o si se altera el formato de texto del comprobante impreso. Para resolver este problema de diseño, se propone sacar la lógica que construye el texto del comprobante y la operación de la multa a otras clases especializadas. De esta manera, la clase *Prestamo* se enfocará exclusivamente en gestionar el estado de la transacción.